

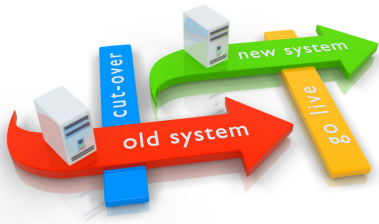
Introduction to Software Change (SB5-MAI)

Jan Corfixen Sørensen

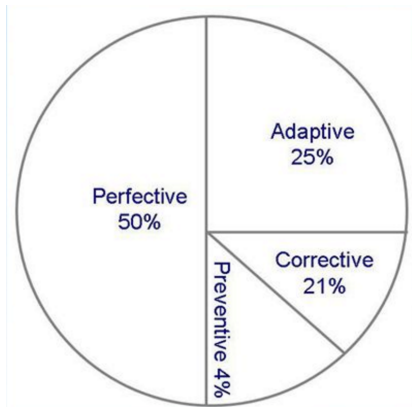
University of Southern Denmark

Software Change

- ▶ Software change (SC) is the process of adding new functionality to existing code.
- ▶ Foundation of software evolution, servicing.



Characteristics of SC



- ▶ Lientz and Swanson:
 - ▶ Perfective (Credit card)
 - ▶ Adaptive (Y2K)
 - ▶ Corrective (Bugs)
 - ▶ Preventive (Improve code structure)

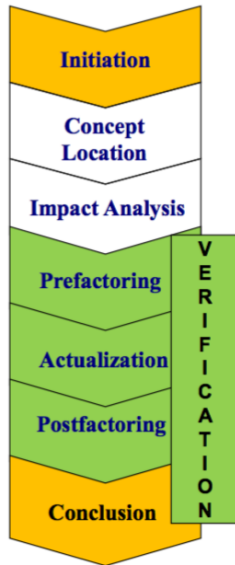
Impact on Functionality

- ▶ Incremental
 - ▶ adding new functionality
- ▶ Contraction
 - ▶ removing obsolete functionality
- ▶ Replacement
 - ▶ replacing existing functionality
- ▶ Refactoring
 - ▶ changing software structure without changing behavior

Phased model of SC

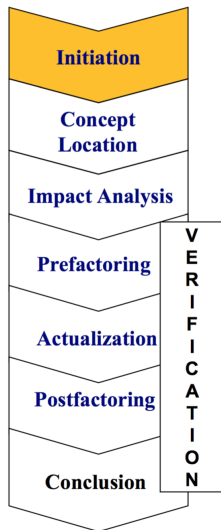
- ▶ Main topic of this course
- ▶ Preview of Phases of SC

Interactions with the world
SC design
SC Implementation

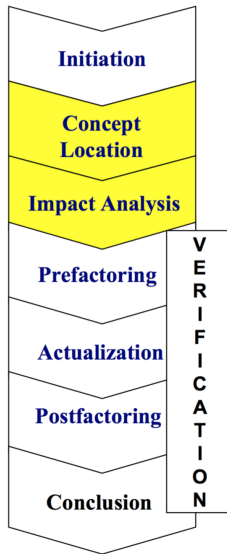
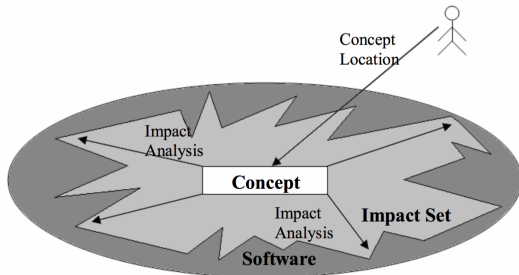


Initiation

- ▶ SC starts by a change request
- ▶ Requirements
 - ▶ software bug
 - ▶ an enhancement
 - ▶ improvement
- ▶ Prioritization of change requests, etc.

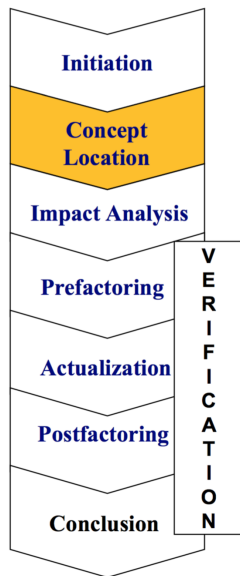


Design



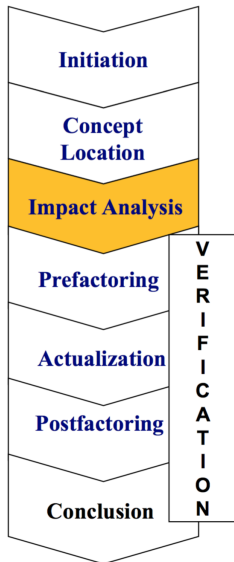
Concept Location

- ▶ Concepts are extracted from change request
- ▶ Extracted concepts are located in the code and used as a starting point of SC



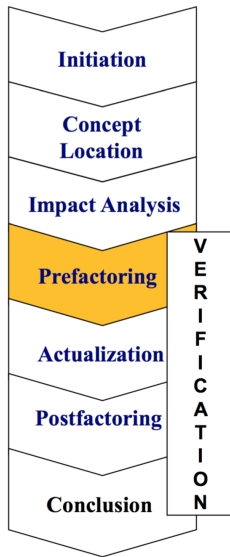
Impact Analysis

- ▶ Determine strategy and impact of change
- ▶ Classes identified in concept location make up the initial impact set
- ▶ Class dependencies are analyzed, and impacted classes are added to the impact set



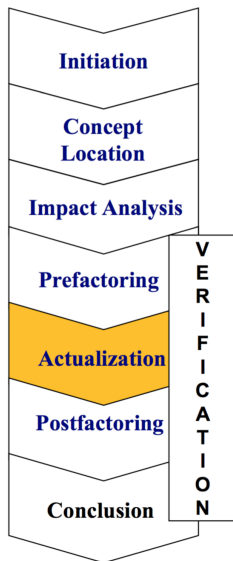
Prefactoring

- ▶ Opportunistic refactoring that localizes (minimizes) impact of SC on software
- ▶ Extract Class (Fowler)
 - ▶ gather fields, methods, and code snippets into a new component class
- ▶ Extract Superclass
 - ▶ create new abstract class



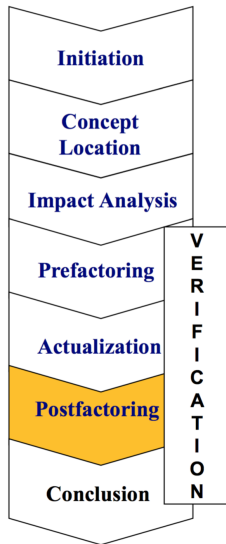
Actualization

- ▶ Creates new code
- ▶ Plugs it into the old code
- ▶ Visit neighboring classes and update them
 - ▶ change propagation
 - ▶ ripple effect



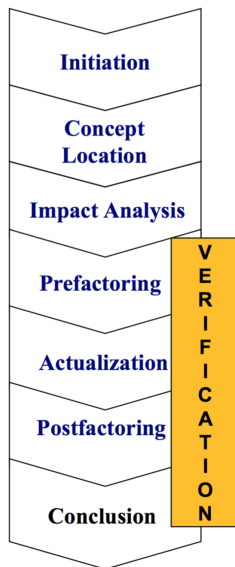
Postfactoring

- ▶ Eliminate any anti-patterns that may have been introduced
 - ▶ long method
 - ▶ after added functionality, some methods may be doing too much
 - ▶ bloated class
 - ▶ after added functionality, a class may be too large



Verification

- ▶ Guarantees correctness of the change
- ▶ Testing
 - ▶ functional
 - ▶ unit
 - ▶ structural
- ▶ Walkthroughs



Conclusion

- ▶ Commit finished code into version control
- ▶ Build the new baseline
- ▶ Prepare for the next change

